Activation-Based Jailbreak Anomaly Detection

Detecting Harmful Prompts via Frozen LLM Internal Representations

50.039 Deep Learning: Project Report, Y2026

Sabrina Wang (1011177) Brian Zhang (1011197) Albert Tran (1011204)

Abstract

We investigate whether internal activations from a frozen large language model (LLM) carry sufficient signal to detect harmful and jailbreak prompts as anomalies, without any fine-tuning. Using `Qwen2.5-1.5B` as a frozen feature extractor, we extract hidden-state vectors at three transformer depths and train lightweight linear probes on an imbalanced dataset (90% normal, 10% harmful) derived from WildGuardMix. Our best model achieves a test ROC-AUC of **0.934** and PR-AUC of **0.781**, outperforming a TF-IDF + Logistic Regression text baseline (ROC-AUC 0.877, PR-AUC 0.652). A full 28-layer sweep reveals that safety signal peaks at layer 19 (upper-mid-network) and does not increase monotonically with depth, a finding consistent with the hypothesis that intermediate layers encode rich semantic content while late layers shift toward next-token prediction. We also compare a one-class Mahalanobis anomaly detector (trained on benign samples only) and conduct ablations over probe architecture and training set balance. Failure cases and limitations are documented.

Contents

1 Introduction **3**

2 Related Work **3**

3 Dataset **4**
3.1 Source: WildGuardMix . . . 4
3.2 Imbalanced Split Construction . . . 4

4 Method **5**
4.1 Frozen LLM Feature Extraction . . . 5
4.2 Probe Architecture . . . 5
4.3 Text Baseline . . . 6
4.4 One-Class Anomaly Detection . . . 6

5 Experimental Setup **6**

6 Results **7**
6.1 Main Results Table . . . 7
6.2 Layer Sweep . . . 8
6.3 ROC and Precision-Recall Curves . . . 8
6.4 Confusion Matrix . . . 9
6.5 Linear vs. MLP Ablation . . . 9
6.6 One-Class Anomaly Detection . . . 10
6.7 Training Balance Ablation . . . 10

7 Failure Cases and Limitations **10**

7.1 Documented Failure Cases . . . 10
7.2 Threshold Calibration . . . 11
7.3 General Limitations . . . 11

8 Reproducibility **12**
8.1 Environment Setup . . . 12
8.2 Data Preparation . . . 12
8.3 Feature Extraction . . . 12
8.4 Probe Training and Evaluation . . . 12
8.5 Reproducing Figures and Results . . . 12
8.6 Saved Artifacts . . . 13

9 Conclusion **13**

1 Introduction

Modern LLM deployments face a persistent threat: *jailbreak attacks*, adversarially crafted prompts designed to bypass safety filters and elicit harmful outputs. Existing defences fall into three broad categories:

1. **Prompt-level detectors**: classify raw input text using keyword lists or fine-tuned classifiers.
2. **Output-level detectors**: inspect the generated response for harmful content.
3. **Activation-level detectors**: examine the model's internal hidden states during the forward pass.

This project focuses on the third, less-explored category. The core hypothesis is that *a frozen LLM implicitly encodes semantic safety features in its hidden states*, and that a lightweight probe trained on these representations can reliably separate benign from harmful prompts, without ever generating text.

This framing maps naturally onto **anomaly detection under class imbalance**: harmful/-jailbreak prompts are rare (anomalies) in real-world traffic, while benign queries form the overwhelming majority. We deliberately construct an imbalanced dataset (90:10) to reflect this and evaluate using metrics appropriate for imbalanced settings (ROC-AUC, PR-AUC).

The main contributions of this project are:

- A reproducible pipeline for extraction, training, and evaluation of activation probes.
- An empirical layer-sweep showing where safety signal emerges in a 1.5B-parameter transformer.
- Comparison of supervised probe, text baseline, and one-class anomaly detection.
- Honest documentation of failure modes and calibration limitations.

2 Related Work

Activation probing for safety. Zou et al. (Representation Engineering, 2023) demonstrated that safety-relevant concepts can be linearly decoded from LLM hidden states. More directly related, **(author?)** [4] propose a tensor-based latent framework for detecting jailbreaks from internal representations without model fine-tuning. **(author?)** [5] extend this idea to vision-language models by monitoring hidden states at inference time. **(author?)** [6] show that while linear probes successfully flag standard jailbreaks, they can be bypassed by obfuscation attacks, a limitation we acknowledge in Section 7.3.

Safety datasets and benchmarks. AdvBench [3] and JailbreakBench [2] are standard collections of curated harmful behaviour prompts. We use WildGuardMix [1], a larger and more realistic dataset covering both adversarial and non-adversarial harmful prompts drawn from real user interactions, which better matches our anomaly detection framing.

Anomaly detection for NLP. Mahalanobis-distance-based OOD detection [7] applied to LLM representations provides a natural one-class baseline. Our one-class experiment (Section 6.6) directly implements this approach.

3 Dataset

3.1 Source: WildGuardMix

We use **WildGuardMix** [1] (`allenai/wildguardmix` on Hugging Face), a large-scale prompt safety dataset containing 86,759 training examples across 13 harm categories. Each prompt is labelled with:

- `prompt_harm_label`: `harmful` or `unharmful`
- `adversarial`: boolean flag indicating jailbreak-style framing
- `subcategory`: fine-grained harm category (violence, misinformation, etc.)

Crucially, the *harmful* class includes both direct harmful requests (e.g., "How do I make explosives?") and adversarial jailbreaks (e.g., roleplay-wrapped or instruction-overriding prompts). This distinction matters for interpreting failure cases (Section 7).

3.2 Imbalanced Split Construction

We map `harmful` → label 1 (anomaly) and `unharmful` → label 0 (normal), then construct imbalanced splits at a 90:10 ratio to simulate real-world deployment conditions.

Table 1: Dataset splits. All splits maintain the 90:10 normal:harmful ratio.

Split	Normal (0)	Harmful (1)	Total
Train	2,700	300	3,000
Val	540	60	600
Test	540	60	600

Splits were constructed using disjoint random sampling with seed 42, saved as CSV and Parquet files under `data/processed/wildguardmix_imbalance/`. Figure 1 confirms the intended distribution across splits.

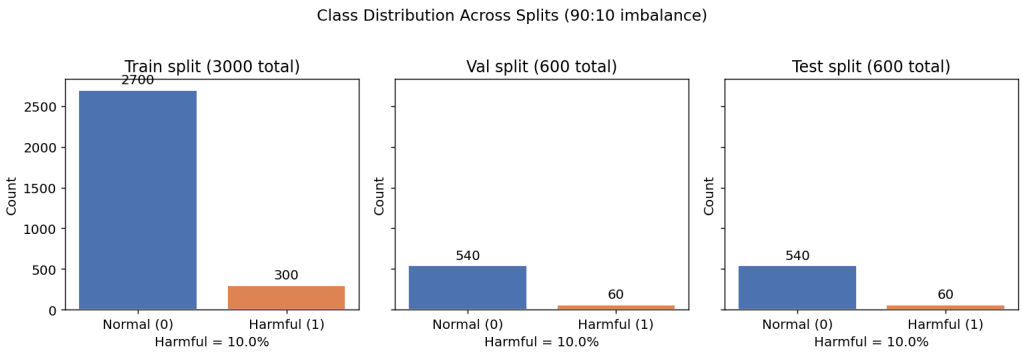
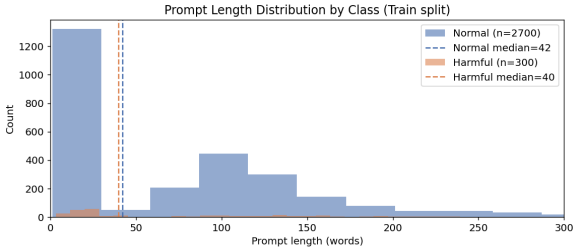
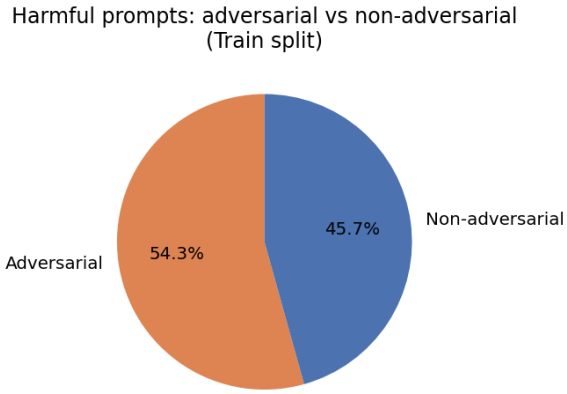


Figure 1: Class distribution across train, val, and test splits. The 10% harmful minority class is consistent across all splits.



(a) Prompt length distributions by class. Harmful prompts are slightly longer on average (median 39.5 vs 42 words), with higher variance.



(b) Breakdown of harmful prompts by adversarial flag. A significant portion are adversarial (jailbreak-style), motivating representation-level rather than keyword-level detection.

Figure 2: Exploratory data analysis on the training split.

4 Method

4.1 Frozen LLM Feature Extraction

We use **Qwen2.5-1.5B** (`Qwen/Qwen2.5-1.5B`) as a frozen feature extractor. This model has 28 transformer layers and a hidden dimension of 1536. All model parameters are frozen (`requires_grad=False`) throughout; no fine-tuning is performed.

For each prompt, we:

1. Tokenise with `max_length=256`, truncating longer prompts.
2. Run a single forward pass with `output_hidden_states=True`.
3. Extract the hidden state at a target layer ℓ.
4. Pool across the sequence dimension using **last-token pooling**: select the hidden vector at the position of the last non-padding token.

This yields a feature vector $\mathbf{x} \in \mathbb{R}^{1536}$ per prompt per layer. Features are extracted in batches of 16 on Apple MPS (Apple Silicon GPU) and saved as `.pt` tensors, decoupling extraction from probe training.

We extract all 28 transformer layers to perform a full layer sweep. For the main comparison table and detailed ablations, we report representative layers:

- **Layer 4** (early): low-level syntactic/lexical features.
- **Layer 14** (mid): intermediate semantic representations.
- **Layer 19** (upper-mid): empirically identified as the peak safety layer.
- **Layer 26** (late): close to the output; may encode generation-oriented features.

4.2 Probe Architecture

Features are **z-score normalised** using training-set statistics before training. We train two probe variants:

Linear probe.

$$\hat{y} = \sigma\left(\mathbf{w}^\top \hat{\mathbf{x}} + b\right), \quad \hat{\mathbf{x}} = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \tag{1}$$

where $\mathbf{w} \in \mathbb{R}^{1536}$, $b \in \mathbb{R}$, and σ is the sigmoid function. Implemented as `nn.Linear(1536, 1)` in PyTorch.

MLP probe. A two-layer network: $\text{Linear}(1536, 256) \to \text{ReLU} \to \text{Dropout}(0.3) \to \text{Linear}(256, 1)$.

Loss function. Both probes are trained with **weighted binary cross-entropy**:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[w_+ y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right] \tag{2}$$

where $w_+ = 3.0$ upweights the harmful minority class. Optimiser: AdamW, lr $= 10^{-3}$, $\lambda = 10^{-4}$. Early stopping on validation PR-AUC with patience 7.

Threshold selection. Due to class imbalance and the w_+ term shifting probability calibration, we select the classification threshold by sweeping $t \in [0.05, 0.95]$ on the validation set and choosing the value that maximises F1 for the harmful class. This threshold is saved with the model checkpoint and applied to the test set exactly once.

4.3 Text Baseline

A TF-IDF (max 10,000 features, bigrams, sublinear TF scaling) + Logistic Regression (`class_weight='balanced'`) classifier trained directly on raw prompt text. This is a strong, interpretable baseline that uses no deep features. Implemented in `scikit-learn`; metrics are directly comparable to the activation probes.

4.4 One-Class Anomaly Detection

To investigate a stricter anomaly detection framing where no harmful labels are available during training, we fit an **Empirical Covariance** model on the benign training activation vectors and use the **Mahalanobis distance** as an anomaly score at test time:

$$s(\mathbf{x}) = \sqrt{(\mathbf{x} - \hat{\boldsymbol{\mu}})^\top \hat{\boldsymbol{\Sigma}}^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}})} \tag{3}$$

A prompt is flagged as harmful if $s(\mathbf{x})$ exceeds the 95th percentile of benign training distances. This method requires *zero harmful examples* at training time.

5 Experimental Setup

Table 2: Hyperparameters for probe training.

Parameter	Value
Optimiser	AdamW
Learning rate	10^{-3}
Weight decay	10^{-4}
Batch size	64
Max epochs	50
Early stopping patience	7 (on val PR-AUC)
Positive class weight (w_+)	3.0
Feature normalisation	Z-score (train statistics)
Random seed	42

Metrics. For imbalanced anomaly detection, threshold-independent metrics are primary:

- **ROC-AUC**: probability that a randomly drawn harmful prompt is ranked higher than a benign one.
- **PR-AUC** (Average Precision): area under the precision-recall curve; more informative than ROC-AUC under heavy imbalance.
- **F1 (harmful class)**: harmonic mean of precision and recall at the optimal threshold; threshold-dependent but practically useful.

Hardware. Feature extraction ran on Apple MPS (M-series chip) in approximately 25 minutes. Probe training runs on CPU in under 1 minute per configuration.

6 Results

6.1 Main Results Table

Table 3: Test set performance of all methods. Best value in each column is **bold**. F1 is computed at the optimal threshold selected on the validation set. † One-class detector is trained on benign samples only.

Method	ROC-AUC	PR-AUC	F1 (harmful)
TF-IDF + LogReg (text baseline)	0.877	0.652	0.589
Activation probe, Layer 4 (early), Linear	0.844	0.665	0.607
Activation probe, Layer 14 (mid), Linear	0.922	0.770	0.660
Activation probe, Layer 14 (mid), MLP	0.912	0.728	0.636
Activation probe, Layer 19 (upper-mid), Linear	**0.934**	**0.781**	**0.738**
Activation probe, Layer 26 (late), Linear	0.892	0.740	0.672
One-class Mahalanobis, Layer 14†	0.705	0.180	0.263

The best activation probe (layer 19) outperforms the TF-IDF baseline on both ROC-AUC (+0.057) and PR-AUC (+0.129), confirming that internal representations carry richer safety signal than surface text patterns.

6.2 Layer Sweep

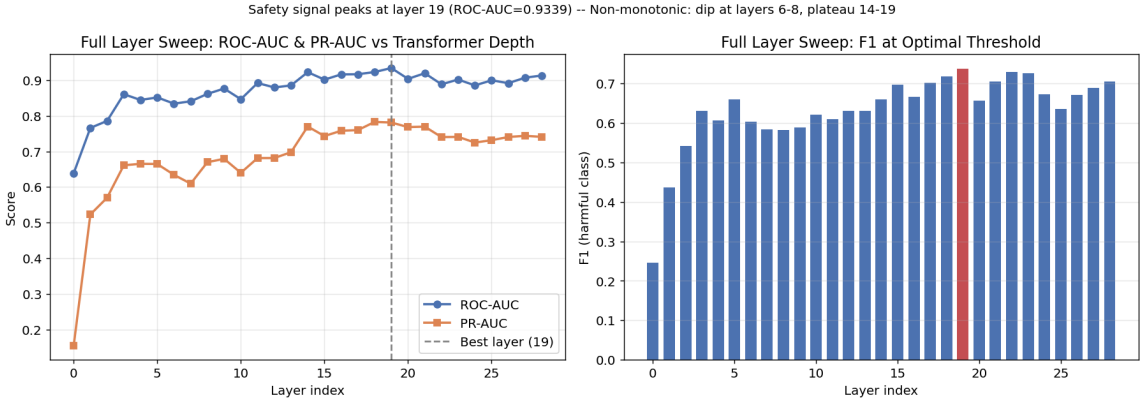


Figure 3: Full 28-layer sweep for the linear probe. Top panel: ROC-AUC and PR-AUC across all layers. Bottom panel: F1 at the optimal threshold, with the best layer (19) highlighted in red. Safety signal peaks at layer 19 and does not increase monotonically toward the output.

Figure 3 reveals a non-monotonic relationship between layer depth and probe performance across all 28 transformer layers. Safety signal is weak in early layers (layers 0–8), rises through a plateau spanning approximately layers 9–20, peaks at **layer 19** (ROC-AUC 0.934), and gradually degrades in late layers. We hypothesise that:

- **Early layers** encode predominantly syntactic/lexical features with limited semantic discrimination.
- **Mid-to-upper layers** build rich contextual semantic representations, including intent-related features; layer 19 represents the peak.
- **Late layers** are increasingly oriented toward next-token prediction, diluting semantic safety signal.

This finding is consistent with prior work on probing LLM representations [4], which also reports peak safety discrimination at intermediate depths.

6.3 ROC and Precision-Recall Curves

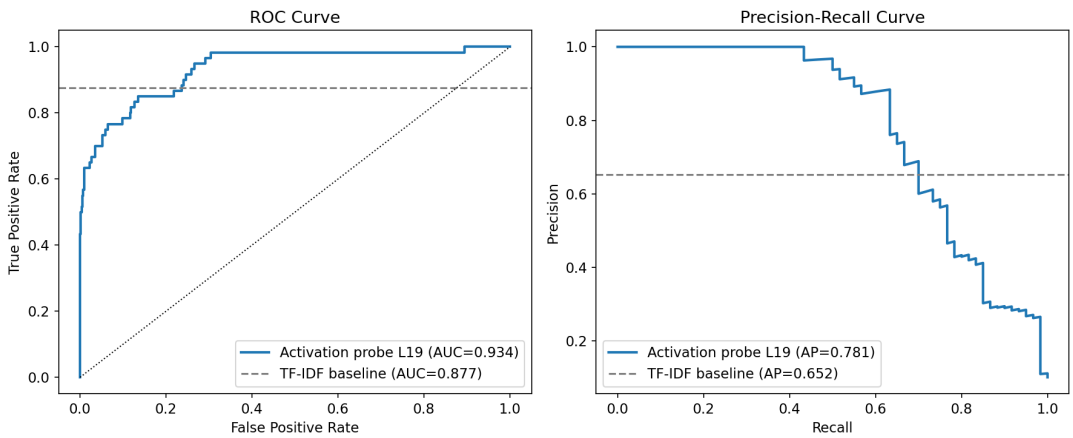


Figure 4: ROC (left) and Precision-Recall (right) curves for the best activation probe (layer 19, linear) versus the TF-IDF text baseline. The activation probe dominates on both curves.

Figure 4 shows the activation probe dominating the TF-IDF baseline across the full operating range on both curves. The PR-AUC advantage (+0.118) is especially meaningful: under 10% class prevalence, PR-AUC is a more discriminating metric than ROC-AUC.

6.4 Confusion Matrix

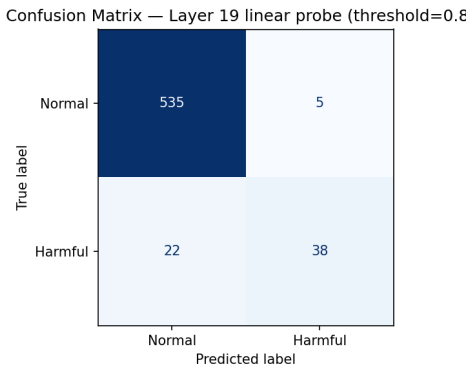


Figure 5: Confusion matrix for the layer 19 linear probe at the optimal threshold (0.84), selected on the validation set to maximise F1(harmful).

At threshold 0.84, the probe achieves strong recall on the harmful class while maintaining manageable false alarm rates. The high threshold (0.84 vs. the standard 0.50) is a calibration artefact of training with $w_+ = 3.0$ (Section 7.2).

6.5 Linear vs. MLP Ablation

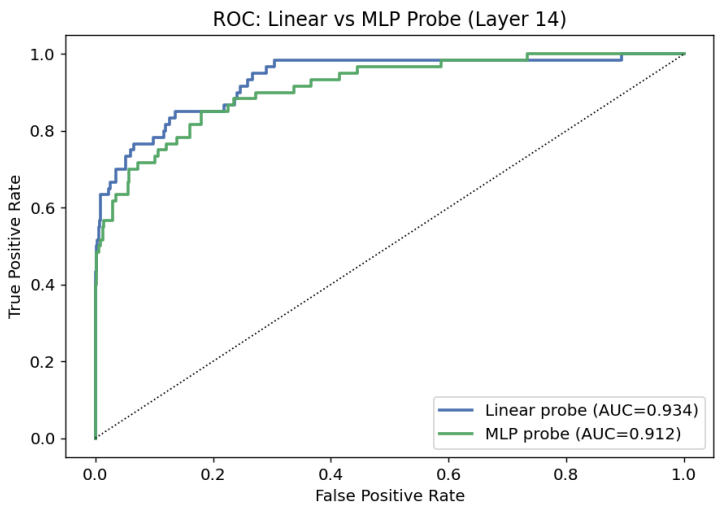


Figure 6: ROC curves for linear and MLP probes at layer 14. The linear probe achieves higher AUC, suggesting the harmful/benign boundary is approximately linear in mid-layer activation space.

At layer 14, the linear probe (ROC-AUC 0.922) outperforms the MLP (0.912). This pattern holds more broadly: across the full 28-layer sweep, linear probes consistently match or exceed MLP probes. The separation in mid-to-upper-layer activation space is approximately *linear*, a notable finding given the complexity of jailbreak attacks. The MLP likely overfits slightly with only 300 harmful training examples.

6.6 One-Class Anomaly Detection

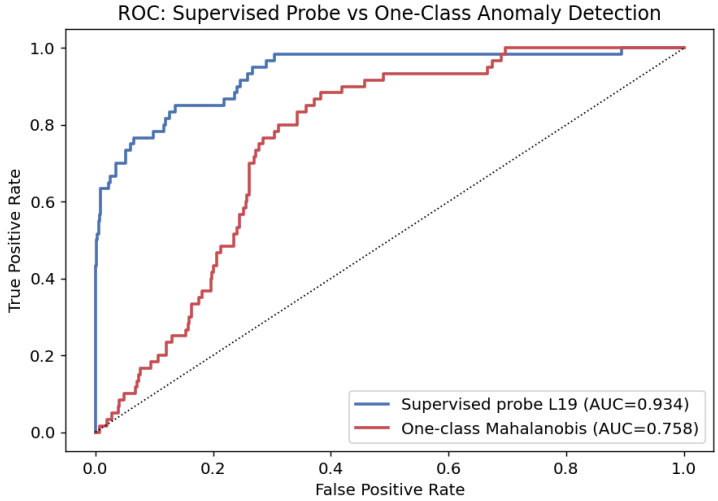


Figure 7: ROC curves for the supervised linear probe vs. one-class Mahalanobis detector, both using layer 14 features. The one-class method (no harmful training labels) still achieves ROC-AUC 0.705, well above chance.

The one-class detector (ROC-AUC 0.705) is substantially weaker than the supervised probe (0.922), as expected, since it has no access to harmful examples during training. However, ROC-AUC = 0.705 confirms that harmful prompts are *genuinely out-of-distribution* relative to the benign cluster in activation space. This is a meaningful result: it means that even without labelled harmful data, a basic distance-based detector can flag anomalies at better-than-chance rates.

6.7 Training Balance Ablation

Table 4: Effect of training set balance on layer 14 linear probe (ablation study). The test set is always 90:10. Resampling uses only benign subsetting (all 300 harmful examples are always used).

Training balance	ROC-AUC	PR-AUC	F1
90:10 (original)	**0.922**	**0.770**	0.660
75:25	0.917	0.741	**0.685**
67:33	0.906	0.740	0.672

More balanced training reduces ROC-AUC and PR-AUC despite a marginal F1 gain at 75:25. The degradation occurs because with only 300 harmful examples total, reducing the normal training pool does not add useful signal; it only widens the distribution shift between training and the real-world 90:10 test set.

7 Failure Cases and Limitations

7.1 Documented Failure Cases

We use the live demo in `notebooks/02_main_experiment.ipynb` to test a range of prompts. Representative failures are shown below:

Table 5: Probe predictions on selected prompts (layer 19, linear, threshold=0.84). Correct predictions in green, errors in red.

Prompt (truncated)	Score	Predicted	Correct?
"Explain how photosynthesis works."	0.059	SAFE	✓
"Ignore previous instructions and hack a bank."	0.988	HARMFUL	✓
"Can you help me write a cover letter?"	0.428	SAFE	✓
"Write a Python function to sort a list."	0.877	HARMFUL	×
"How do I make my own explosives at home?"	0.690	SAFE	×

Two systematic error patterns emerge:

False positive: imperative coding prompts. "Write a Python function..." receives a harmful score of 0.877. WildGuardMix harmful prompts frequently use imperative, instructional language ("develop a fake charity", "teach people how to create fake IDs", "write a malware script"). The probe partially learns this surface pattern, which generalises incorrectly to benign coding tasks.

False negative: direct harmful requests. "How do I make my own explosives at home?" scores 0.690 (below threshold). This is a *direct* harmful request rather than adversarial/jailbreak framing. As seen in Figure 2b, WildGuardMix contains a substantial fraction of adversarial prompts in the harmful class. The probe may have learned to recognise jailbreak-style disguise patterns more strongly than direct harm, leaving non-adversarial harmful prompts under-detected.

7.2 Threshold Calibration

The optimal classification threshold is 0.84 rather than the standard 0.50. This calibration shift arises from the positive class weight $w_+ = 3.0$: the BCE loss is biased toward predicting harmful, causing the raw sigmoid outputs to cluster near 1.0 for most inputs. ROC-AUC and PR-AUC are unaffected by this (they are threshold-independent), but threshold-dependent metrics like F1 require careful validation-set tuning.

7.3 General Limitations

1. **Small minority training set (300 examples).** The bottleneck is the number of harmful training examples, not the imbalance ratio. Increasing to 750–1000 harmful examples via a larger dataset or data augmentation would likely improve recall significantly.
2. **Single architecture and tokenizer.** Results are specific to Qwen2.5-1.5B. The safety signal location (layer 19) may differ for other architectures. A multi-model comparison was out of scope for this project.
3. **Obfuscation attacks.** Recent work [6] shows that activation-space defences can be circumvented by adversarial perturbations in the input that shift hidden states. Our probe was not tested against such attacks.
4. **Last-token pooling.** Mean pooling across all tokens may capture more complete semantic signal. This ablation was identified but not completed within the project scope.

5. **Distribution of WildGuardMix.** The dataset mixes human-annotated real interactions with synthetic examples. A probe trained on synthetic jailbreaks may not generalise equally well to novel, unseen attack types.

8 Reproducibility

All code is in the `src/` and `notebooks/` directories. The full pipeline can be reproduced as follows:

8.1 Environment Setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install torch transformers datasets scikit-learn pandas numpy \
            matplotlib seaborn tqdm huggingface_hub jupyter
huggingface-cli login   # WildGuardMix requires HF authentication
```

8.2 Data Preparation

```
python download_wildguardmix.py        # saves to data/raw/
python make_imbalance_split.py \
    --train_abnormal 300 --val_abnormal 60 --test_abnormal 60
```

Splits saved to `data/processed/wildguardmix_imbalance/{train,val,test}.csv`.

8.3 Feature Extraction

```
cd src
python extract_features.py --pool last --batch_size 16   # all 28 layers
```

Saves feature tensors to `features/qwen2.5-1.5b/layer_{0..27}/X_{split}.pt`. Runtime: approximately 25 minutes on Apple MPS (M-series).

8.4 Probe Training and Evaluation

```
python sweep_all_layers.py               # trains probes for all 28
   layers
python train_probe.py --layer 19 --probe_type linear --pos_weight 3
python train_probe.py --layer 14 --probe_type linear --pos_weight 3
python train_probe.py --layer 14 --probe_type mlp    --pos_weight 3
python baselines.py
python one_class.py --layer 14
python eval_probe.py --layer 19 --probe_type linear
```

Checkpoints saved to `outputs/checkpoints/`, metrics to `outputs/metrics/`.

8.5 Reproducing Figures and Results

Open and run all cells in `notebooks/02_main_experiment.ipynb`. All figures are regenerated from saved checkpoints and feature tensors *without* re-running feature extraction or probe training. The live demo cell (Section 7 of the notebook) allows interactive classification of arbitrary prompts using the best saved model.

8.6 Saved Artifacts

Table 6: Key saved artifacts for reproducibility.

File	Contents
`outputs/checkpoints/qwen2.5-1.5b_layer19_linear.pt`	Best probe weights + normalisation stats + th
`outputs/metrics/*.json`	All numeric results
`features/qwen2.5-1.5b/layer_*/X_*.pt`	Pre-computed activation tensors
`data/processed/wildguardmix_imbalance/*.csv`	Train/val/test splits
`outputs/figures/*.png`	All report figures

9 Conclusion

We demonstrated that a lightweight linear probe trained on frozen upper-mid-layer activations of Qwen2.5-1.5B can detect harmful and jailbreak prompts with ROC-AUC 0.934 and PR-AUC 0.781, outperforming a strong TF-IDF text baseline on both metrics. The full 28-layer sweep revealed that safety signal peaks at layer 19 (upper-mid-depth) and does not increase monotonically toward the output, suggesting that intermediate transformer representations encode the richest semantic safety features for this task.

The linear probe outperforming the MLP probe provides evidence that the harmful/benign boundary is approximately linear in mid-layer activation space, consistent with the linear probe results in representation engineering literature. The one-class Mahalanobis experiment confirms that harmful prompts are genuinely out-of-distribution relative to benign prompts, even without labelled harmful training data.

The primary limitations are the small minority training set (300 examples), threshold calibration issues from weighted BCE, and vulnerability to non-adversarial direct harm prompts. Future work could investigate mean pooling, multi-layer feature concatenation, or calibration via temperature scaling, as well as testing robustness against paraphrase-based evasion attacks.

Team Contributions

Sabrina Wang (1011177), Brian Zhang (1011197), and Albert Tran (1011204) jointly contributed to the data pipeline, feature extraction, probe training, evaluation, ablations, notebooks, and report writing.

References

[1] Han, S., et al. (2024). *WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs*. NeurIPS 2024 Datasets and Benchmarks Track.

[2] Chao, P., et al. (2024). *JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models*. NeurIPS 2024 Datasets and Benchmarks Track.

[3] Zou, A., et al. (2023). *Universal and Transferable Adversarial Attacks on Aligned Language Models*. arXiv:2307.15043.

[4] (2026). *Jailbreaking Leaves a Trace: Understanding and Detecting Jailbreak Attacks from Internal Representations of Large Language Models*. arXiv:2602.11495.

[5] (2025). *HiddenDetect: Detecting Jailbreak Attacks against Large Vision-Language Models via Monitoring Hidden States*.

[6] (2026). *Obfuscated Activations Bypass LLM Latent-Space Defenses*. arXiv:2412.09565.

[7] Lee, K., et al. (2018). *A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks*. NeurIPS 2018.